

Android Jetpack Compose Best Practices Guide

Design • Coding Standards • Memory Management • Security • Accessibility •
ProGuard • Performance • Error Handling • CI/CD

A comprehensive reference for building robust, performant, and accessible Android apps with
Jetpack Compose.

Table of Contents

1. Design Principles

- 1.1 Material Design 3
- 1.2 Theming & Color System
- 1.3 Typography
- 1.4 Layout & Spacing
- 1.5 Dark/Light Theme

2. Architecture & State Management

- 2.1 MVVM Pattern
- 2.2 State Hoisting
- 2.3 ViewModel Integration
- 2.4 Side Effects
- 2.5 UDF Pattern

3. Performance & Memory Management

- 3.1 Recomposition Optimization
- 3.2 remember & derivedStateOf
- 3.3 Lazy Lists
- 3.4 Image Loading
- 3.5 Memory Leak Prevention

4. Mobile Best Practices

- 4.1 Navigation
- 4.2 Responsive/Adaptive UI
- 4.3 Orientation Handling
- 4.4 System Bars & Edge-to-Edge

5. ProGuard / R8

- 5.1 Setup
- 5.2 Compose-Specific Rules
- 5.3 Obfuscation & Shrinking
- 5.4 Debugging

6. Security

- 6.1 EncryptedSharedPreferences
- 6.2 Biometric Auth
- 6.3 Network Security Config
- 6.4 Secure Coding Patterns

7. Accessibility

- 7.1 Semantics
- 7.2 Content Descriptions
- 7.3 Touch Target Size
- 7.4 TalkBack Support

8. Testing

- 8.1 Unit Testing with StateFlow
- 8.2 Compose UI Testing

8.3 Screenshot Testing

9. Dependency Injection (Hilt)

10. Animations

10.1 animate*AsState

10.2 AnimatedVisibility

10.3 Transition API

11. Networking & Data

11.1 Retrofit + Kotlin Flow

11.2 Room Database

11.3 DataStore

12. Coding Standards

12.1 Kotlin Naming Conventions

12.2 Composable Naming Rules

12.3 File & Package Structure

12.4 Modifier Convention

12.5 Parameter Ordering

12.6 Preview Annotations

12.7 Code Formatting & Linting (ktlint / detekt)

13. Kotlin Best Practices

13.1 Null Safety

13.2 Scope Functions

13.3 Extension Functions

13.4 Sealed Classes & When

13.5 Coroutines & Flow Patterns

13.6 Data & Value Classes

14. Error Handling & Logging

14.1 Result / Either Pattern

14.2 Global Error Handling

14.3 Structured Logging

14.4 Crash Reporting

15. Gradle & Build Best Practices

15.1 Version Catalogs

15.2 Build Variants

15.3 Modularisation

15.4 Build Performance

16. CI/CD for Android

16.1 GitHub Actions Workflow

16.2 Static Analysis

16.3 Automated Testing

16.4 Play Store Deployment

17. Battery Life Optimization

17.1 WorkManager for Deferrable Work

17.2 Doze & App Standby

17.3 Foreground Services

17.4 Efficient Wake Locks & Alarms

17.5 Network & Sensor Batching

18. Secure Communication

18.1 TLS / HTTPS Enforcement

18.2 Certificate Pinning

18.3 OAuth 2.0 & Token Management

18.4 SSL Debugging

18.5 Encrypted Payloads

19. Exchanging Data Between Apps

19.1 Intents & Intent Filters

19.2 Pending Intents

19.3 AIDL & Bound Services

19.4 Broadcast Receivers

19.5 File Sharing via FileProvider

20. Content Providers

20.1 Built-in Content Providers

20.2 Custom Content Provider

20.3 Querying a Provider in Compose

20.4 CursorLoader & Paging

20.5 Permissions & Security

1. Design Principles

Jetpack Compose adopts Material Design 3 (Material You) as its primary design language. Following Google's design guidelines ensures visual consistency, predictable user experience, and built-in accessibility compliance across the Android ecosystem.

1.1 Material Design 3

Material 3 introduces dynamic color, expressive motion, and adaptive components. Always scaffold your app inside `MaterialTheme` so every child composable inherits tokens for color, typography, and shapes automatically.

Kotlin – MaterialTheme scaffold

```
@Composable
fun MyApp() {
    MyAppTheme { // wraps MaterialTheme
        Surface(
            modifier = Modifier.fillMaxSize(),
            color = MaterialTheme.colorScheme.background
        ) {
            HomeScreen()
        }
    }
}
```

- Use M3 components (`Button`, `Card`, `TopAppBar`) over custom views.
- Respect elevation tokens; avoid hard-coded dp shadow values.
- Use Shape tokens (`MaterialTheme.shapes.medium`) for consistency.

1.2 Theming & Color System

Define a single source of truth for colors via `ColorScheme`. Use `dynamicColorScheme` on Android 12+ to respect wallpaper-based colors.

Kotlin – Dynamic color theme

```
@Composable
fun MyAppTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    dynamicColor: Boolean = true,
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            val ctx = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(ctx)
            else dynamicLightColorScheme(ctx)
        }
        darkTheme -> DarkColorScheme
        else -> LightColorScheme
    }
    MaterialTheme(
        colorScheme = colorScheme,
```

```
typography = AppTypography,
content = content
)
}
```

1.3 Typography

Compose Typography uses sp units for font sizes and supports responsive scaling. Define a project-wide Typography object and never hard-code TextStyle inside composables.

Kotlin – Custom Typography

```
val AppTypography = Typography(
    headlineLarge = TextStyle(
        fontFamily = FontFamily.SansSerif,
        fontWeight = FontWeight.Bold,
        fontSize = 32.sp,
        lineHeight = 40.sp
    ),
    bodyMedium = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.Normal,
        fontSize = 14.sp,
        lineHeight = 20.sp,
        letterSpacing = 0.25.sp
    )
)

// Usage
Text(
    text = "Hello Compose",
    style = MaterialTheme.typography.headlineLarge
)
```

1.4 Layout & Spacing

Use a spacing design token system to maintain consistent padding/margins. Compose's Modifier chain is the primary layout mechanism — avoid nesting unnecessary boxes.

Kotlin – Spacing tokens + layout

```
object Spacing {
    val xs = 4.dp
    val sm = 8.dp
    val md = 16.dp
    val lg = 24.dp
    val xl = 32.dp
}

@Composable
fun ProfileCard(name: String, bio: String) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = Spacing.md, vertical = Spacing.sm),
    )
}
```

```

shape = MaterialTheme.shapes.medium,
elevation = CardDefaults.cardElevation(defaultElevation = 4.dp)
) {
Column(modifier = Modifier.padding(Spacing.md)) {
Text(name, style = MaterialTheme.typography.titleMedium)
Spacer(Modifier.height(Spacing.xs))
Text(bio, style = MaterialTheme.typography.bodySmall)
}
}
}

```

1.5 Dark / Light Theme

Always support both dark and light modes. Use `MaterialTheme.colorScheme` tokens rather than hard-coded Color values so the UI adapts automatically.

Kotlin – Dark/Light preview

```

@Preview(name = "Light", uiMode = UI_MODE_NIGHT_NO)
@Preview(name = "Dark", uiMode = UI_MODE_NIGHT_YES)
@Composable
fun CardPreview() {
MyAppTheme { ProfileCard("Alex", "Android Engineer") }
}

```

2. Architecture & State Management

Compose works best with a Unidirectional Data Flow (UDF) architecture. The recommended pattern is MVVM with a ViewModel exposing immutable UI state via `StateFlow`, and the composable triggering events back to the ViewModel.

2.1 MVVM Pattern

Kotlin – ViewModel + UiState

```
// State model
data class HomeUiState(
    val items: List<Item> = emptyList(),
    val isLoading: Boolean = false,
    val error: String? = null
)

// ViewModel
@HiltViewModel
class HomeViewModel @Inject constructor(
    private val repo: ItemRepository
) : ViewModel() {

    private val _uiState = MutableStateFlow(HomeUiState())
    val uiState: StateFlow<HomeUiState> = _uiState.asStateFlow()

    fun loadItems() {
        viewModelScope.launch {
            _uiState.update { it.copy(isLoading = true) }
            repo.getItems()
                .onSuccess { items ->
                    _uiState.update { it.copy(items = items, isLoading = false) }
                }
                .onFailure { e ->
                    _uiState.update { it.copy(error = e.message, isLoading = false) }
                }
        }
    }
}
```

2.2 State Hoisting

Move state up to the lowest common ancestor. Stateless composables are easier to test, preview, and reuse. State should be owned by the caller, not the component.

Kotlin – Stateless TextField component

```
// Stateless (reusable, testable)
@Composable
fun EmailInput(
    value: String,
    onValueChange: (String) -> Unit,
    modifier: Modifier = Modifier
)
```



```

) {
OutlinedTextField(
value = value,
onValueChange = onValueChange,
label = { Text("Email") },
modifier = modifier
)
}

// Caller owns the state
@Composable
fun LoginScreen(vm: LoginViewModel = hiltViewModel()) {
val email by vm.email.collectAsStateWithLifecycle()
EmailInput(value = email, onValueChange = vm::onEmailChanged)
}

```

2.3 ViewModel Integration

Kotlin – collectAsStateWithLifecycle

```

@Composable
fun HomeScreen(vm: HomeViewModel = hiltViewModel()) {
val state by vm.uiState.collectAsStateWithLifecycle()

LaunchedEffect(Unit) { vm.loadItems() }

when {
state.isLoading -> CircularProgressIndicator()
state.error != null -> ErrorMessage(state.error!!)
else -> ItemList(state.items)
}
}

```

- Prefer `collectAsStateWithLifecycle` over `collectAsState` — it respects `Lifecycle.State.STARTED`.
- Never pass `ViewModel` instances into child composables; pass state and lambdas instead.

2.4 Side Effects

Use the correct effect handler for each scenario. Misusing effects is a common source of bugs and unnecessary recompositions.

Effect API	Use Case
<code>LaunchedEffect(key)</code>	Coroutine tied to composition lifecycle
<code>DisposableEffect(key)</code>	Setup + cleanup (listeners, sensors)
<code>SideEffect</code>	Sync Compose state to non-Compose code (each recomposition)
<code>rememberCoroutineScope</code>	User-triggered coroutines from callbacks
<code>produceState</code>	Convert non-Compose state source into <code>State<T></code>

Kotlin – DisposableEffect for sensor listener

```

@Composable
fun SensorScreen(sensorManager: SensorManager) {
    DisposableEffect(sensorManager) {
        val listener = MySensorListener()
        sensorManager.registerListener(listener, ...)
        onDispose {
            sensorManager.unregisterListener(listener)
        }
    }
}

```

2.5 Unidirectional Data Flow (UDF)

Kotlin – Events sealed class pattern

```

sealed class HomeEvent {
    data class Search(val query: String) : HomeEvent()
    object Refresh : HomeEvent()
    data class DeleteItem(val id: Int) : HomeEvent()
}

// ViewModel handles events
fun onEvent(event: HomeEvent) = when (event) {
    is HomeEvent.Search -> search(event.query)
    HomeEvent.Refresh -> loadItems()
    is HomeEvent.DeleteItem -> delete(event.id)
}

// Composable dispatches events
Button(onClick = { vm.onEvent(HomeEvent.Refresh) }) {
    Text("Refresh")
}

```

3. Performance & Memory Management

Compose tracks reads of State objects and only recomposes the minimal set of composables that read changed state. Understanding and controlling recomposition scope is the most impactful performance technique available.

3.1 Recomposition Optimization

Kotlin – Stable & Immutable annotations

```
// Mark data classes as @Stable so Compose skips recomposition
// when parameters haven't changed structurally.
@Immutable
data class UserProfile(
    val id: Int,
    val name: String,
    val avatar: String
)

// Use lambda references (not lambdas) to avoid recomposition
@Composable
fun UserCard(profile: UserProfile, onClick: () -> Unit) {
    Card(onClick = onClick) { // stable reference from caller
        Text(profile.name)
    }
}
```

- Prefer @Immutable data classes over mutable ones for Compose parameters.
- Extract repeated lambda bodies into remembered callbacks to prevent restarts.
- Use key() in lazy lists to help Compose identify stable items.
- Profile recompositions in Android Studio Layout Inspector > Recomposition counts.

3.2 remember & derivedStateOf

Kotlin – derivedStateOf to avoid over-recomposition

```
@Composable
fun CartScreen(cartItems: List<CartItem>) {

    // Without derivedStateOf: recomputes on EVERY recomposition
    // val total = cartItems.sumOf { it.price } <- avoid

    // With derivedStateOf: only recalculates when cartItems actually changes
    val total by remember(cartItems) {
        derivedStateOf { cartItems.sumOf { it.price } }
    }

    // Only recomposes Text when total changes
    Text("Total: $$total")
}
```

Kotlin – rememberSaveable for process death survival

```

@Composable
fun SearchBar() {
    var query by rememberSaveable { mutableStateOf("") }
    OutlinedTextField(
        value = query,
        onValueChange = { query = it },
        label = { Text("Search") }
    )
}

```

3.3 Lazy Lists — LazyColumn / LazyRow

Kotlin – LazyColumn with keys and contentType

```

@Composable
fun ItemFeed(items: List<FeedItem>) {
    LazyColumn(
        contentPadding = PaddingValues(16.dp),
        verticalArrangement = Arrangement.spacedBy(8.dp)
    ) {
        items(
            items = items,
            key = { it.id }, // stable identity -> better diffing
            contentType = { it::class.java } // reuse composable slots by type
        ) { item ->
            FeedCard(item)
        }
    }
}

```

- Always provide key= in items/itemsIndexed to prevent unnecessary recompositions on list changes.
- Use contentType to improve composable slot reuse between different item types.
- Avoid placing LazyColumn inside a scrollable Column — causes unbounded height crashes.
- Use LazyListState to programmatically control scroll position and read firstVisibleItemIndex.

3.4 Image Loading

Kotlin – Coil AsyncImage with memory management

```

// build.gradle: implementation("io.coil-kt:coil-compose:2.x")
@Composable
fun AvatarImage(url: String, modifier: Modifier = Modifier) {
    AsyncImage(
        model = ImageRequest.Builder(LocalContext.current)
            .data(url)
            .crossfade(true)
            .diskCachePolicy(CachePolicy.ENABLED)
            .memoryCachePolicy(CachePolicy.ENABLED)
            .size(128, 128) // downsample to display size
            .build(),
        contentDescription = "User avatar",
        contentScale = ContentScale.Crop,
        modifier = modifier
            .size(48.dp)
            .clip(CircleShape)
    )
}

```

```
}
```

- Always specify size() in ImageRequest to avoid loading full-resolution images.
- Prefer disk cache for remote images to reduce network re-fetches.
- Use placeholder() and error() images for graceful fallback states.

3.5 Memory Leak Prevention

Kotlin – Avoiding context leaks

```
// BAD: capturing Activity context in a ViewModel
class BadViewModel(private val context: Context) : ViewModel()

// GOOD: use ApplicationContext or AndroidViewModel
@HiltViewModel
class GoodViewModel @Inject constructor(
    @ApplicationContext private val context: Context
) : ViewModel()

// BAD: retaining composable lambdas beyond their lifecycle
class BadManager {
    var onDataReady: (() -> Unit)? = null // leaks composable
}

// GOOD: use ViewModel events / SharedFlow for one-shot events
private val _events = MutableSharedFlow<UiEvent>()
val events: SharedFlow<UiEvent> = _events.asSharedFlow()
```

- Never store Context, View, or Window references in ViewModel or singletons.
- Cancel coroutines properly — use viewModelScope which auto-cancels on ViewModel.onCleared().
- Unregister listeners in DisposableEffect.onDispose {}.
- Use WeakReference for callbacks in long-lived objects if absolutely necessary.

4. Mobile Best Practices

4.1 Navigation with Navigation Compose

Kotlin – Type-safe navigation (Navigation 2.8+)

```
// Define routes as serializable objects/classes
@Serializable object HomeRoute
@Serializable data class DetailRoute(val itemId: Int)

@Composable
fun AppNavHost(navController: NavHostController) {
    NavHost(navController, startDestination = HomeRoute) {
        composable<HomeRoute> {
            HomeScreen(onItemClick = { id ->
                navController.navigate(DetailRoute(id))
            })
        }
        composable<DetailRoute> { backStackEntry ->
            val route: DetailRoute = backStackEntry.toRoute()
            DetailScreen(itemId = route.itemId)
        }
    }
}
```

- Use a single NavHost at the app level; nest NavHosts sparingly.
- Pass NavController only to top-level screen composables, not deep into the tree.
- Use SavedStateHandle in ViewModel to read nav arguments.
- Prefer popUpTo with inclusive=true to avoid back-stack accumulation.

4.2 Responsive & Adaptive UI

Kotlin – WindowSizeClass adaptive layout

```
@Composable
fun AdaptiveApp(windowSizeClass: WindowSizeClass) {
    when (windowSizeClass.widthSizeClass) {
        WindowWidthSizeClass.Compact ->
            MobileLayout()
        WindowWidthSizeClass.Medium ->
            TabletLayout(showPanel = false)
        WindowWidthSizeClass.Expanded ->
            DesktopLayout()
    }
}

@Composable
fun MobileLayout() {
    Scaffold(
        bottomBar = { BottomNavigationBar() }
    ) { padding ->
        MainContent(Modifier.padding(padding))
    }
}
```

```

}

@Composable
fun DesktopLayout() {
    Row {
        NavigationRail()
        MainContent(Modifier.weight(1f))
        DetailPanel(Modifier.width(340.dp))
    }
}

```

4.3 Configuration Changes & Orientation

Kotlin – Portrait/Landscape layout switching

```

@Composable
fun OrientationAwareScreen() {
    val configuration = LocalConfiguration.current
    if (configuration.orientation == Configuration.ORIENTATION_LANDSCAPE) {
        LandscapeContent()
    } else {
        PortraitContent()
    }
}

```

- Do NOT add `android:configChanges` to the manifest unless you handle it yourself.
- `ViewModel` survives configuration changes; state inside `remember{}` does NOT.
- Use `rememberSaveable` for state that must survive both recomposition and config change.

4.4 Edge-to-Edge & System Bars

Kotlin – Edge-to-edge with `WindowInsets`

```

// In Activity.onCreate()
WindowCompat.setDecorFitsSystemWindows(window, false)

// In composable, respect insets
Scaffold(
    modifier = Modifier
        .fillMaxSize()
        .windowInsetsPadding(WindowInsets.systemBars)
) { padding ->
    Content(Modifier.padding(padding))
}

// Or per-element
Box(
    modifier = Modifier
        .navigationBarPadding()
        .imePadding() // keyboard-aware
) { ... }

```

5. ProGuard / R8

R8 (the successor to ProGuard) performs code shrinking, obfuscation, and optimization. Enabling it in release builds significantly reduces APK size and hardens the app against reverse engineering.

5.1 Setup in build.gradle

Groovy/Kotlin DSL – build.gradle

```
android {
    buildTypes {
        release {
            isMinifyEnabled = true // enable R8 shrinking
            isShrinkResources = true // remove unused resources
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
        debug {
            isMinifyEnabled = false // disable in debug for faster builds
        }
    }
}
```

5.2 Compose-Specific ProGuard Rules

proguard-rules.pro – Compose rules

```
# Keep Compose runtime internals
-keep class androidx.compose.** { *; }
-keepclassmembers class androidx.compose.** { *; }

# Keep @Composable annotated functions
-keep @androidx.compose.runtime.Composable class * { *; }

# Keep Kotlin metadata for reflection
-keepattributes RuntimeVisibleAnnotations
-keepattributes *Annotation*
-keepattributes Signature, InnerClasses, EnclosingMethod

# Kotlin coroutines
-keepclassmembers class kotlinx.coroutines.** { *; }

# Hilt / Dagger
-keep class dagger.hilt.** { *; }
-keep @dagger.hilt.android.HiltAndroidApp class * { *; }
-keep @dagger.hilt.android.AndroidEntryPoint class * { *; }

# Retrofit + OkHttp
-keepattributes Exceptions
-keep class retrofit2.** { *; }
-keep interface retrofit2.** { *; }
```



```
-keepclassmembers,allowshrinking,allowobfuscation interface * {
@retrofit2.http.* <methods>;
}

# Data classes used in serialization
-keep class com.example.app.data.model.** { *; }

# Prevent obfuscation of enums
-keepclassmembers enum * { *; }
```

5.3 Obfuscation & Shrinking Tips

- Always test release builds — R8 can break runtime reflection silently.
- Use `-whyareyoukeeping ClassName` to debug unexpected keeps.
- Use `-printmapping mapping.txt` and upload to Play Console for deobfuscated stack traces.
- Consumer ProGuard rules should be added to library modules via `consumerProguardFiles`.
- Use `@Keep` annotation on individual classes/methods instead of broad `-keep` rules.

Kotlin — @Keep annotation

```
import androidx.annotation.Keep

@Keep // Prevents R8 from removing or renaming this class
data class ApiResponse(
    val status: String,
    val data: List<Item>
)
```

5.4 Debugging R8 Issues

Shell — Retrace deobfuscated stack trace

```
# Using Android SDK retrace tool
retrace.bat mapping.txt stacktrace.txt

# Or via Gradle
./gradlew app:deobfuscate{BuildType}WithSpecificProguardFile
```

6. Security

Security is a layered discipline. For Compose apps, the most critical areas are secure data storage, authentication, network security, and preventing sensitive data exposure in screenshots or logs.

6.1 EncryptedSharedPreferences

Kotlin – AES-256 encrypted preferences

```
// Dependency: androidx.security:security-crypto:1.1.x
val masterKey = MasterKey.Builder(context)
    .setKeyScheme(MasterKey.KeyScheme.AES256_GCM)
    .build()

val prefs = EncryptedSharedPreferences.create(
    context,
    "secure_prefs",
    masterKey,
    EncryptedSharedPreferences.PrefKeyEncryptionScheme.AES256_SIV,
    EncryptedSharedPreferences.PrefValueEncryptionScheme.AES256_GCM
)

// Write
prefs.edit().putString("auth_token", token).apply()

// Read
val token = prefs.getString("auth_token", null)
```

- Never store tokens/passwords in plain SharedPreferences or plain files.
- For larger encrypted data, use Room with SQLCipher or the Android Keystore directly.
- Use DataStore (Preferences or Proto) for new projects — it replaces SharedPreferences.

6.2 Biometric Authentication

Kotlin – BiometricPrompt in Compose

```
@Composable
fun BiometricGate(onAuthSuccess: () -> Unit) {
    val context = LocalContext.current
    val executor = remember { ContextCompat.getMainExecutor(context) }
    val fragment = context as FragmentActivity

    val prompt = remember {
        BiometricPrompt(fragment, executor,
            object : BiometricPrompt.AuthenticationCallback() {
                override fun onAuthenticationSucceeded(
                    result: BiometricPrompt.AuthenticationResult
                ) { onAuthSuccess() }
            })
    }

    Button(onClick = {
        val info = BiometricPrompt.PromptInfo.Builder()
```

```

.setTitle("Unlock App")
.setSubtitle("Use biometric credential")
.setNegativeButton("Cancel")
.build()
prompt.authenticate(info)
}) {
Text("Authenticate")
}
}

```

6.3 Network Security Config

XML – res/xml/network_security_config.xml

```

<?xml version="1.0" encoding="utf-8"?>
<network-security-config>
<!-- Only allow HTTPS in production -->
<base-config cleartextTrafficPermitted="false">
<trust-anchors>
<certificates src="system" />
</trust-anchors>
</base-config>

<!-- Certificate pinning for your API domain -->
<domain-config>
<domain includeSubdomains="true">api.example.com</domain>
<pin-set expiration="2026-01-01">
<pin digest="SHA-256">base64encodedpin==</pin>
<pin digest="SHA-256">backuppin==</pin>
</pin-set>
</domain-config>

<!-- Allow cleartext for debug builds only -->
<debug-overrides>
<trust-anchors>
<certificates src="user" />
</trust-anchors>
</debug-overrides>
</network-security-config>

```

Kotlin – OkHttp certificate pinner (alternative)

```

val okHttpClient = OkHttpClient.Builder()
    .certificatePinner(
        CertificatePinner.Builder()
            .add("api.example.com", "sha256/AAAAAAAAAAAAAAAAAAAAAAAAAAAA=")
            .build()
    )
    .build()

```

6.4 Secure Coding Patterns

Kotlin – FLAG_SECURE to block screenshots of sensitive screens

```

@Composable
fun SecureScreen() {

```

```

val view = LocalView.current
DisposableEffect(Unit) {
    val window = (view.context as Activity).window
    window.addFlags(WindowManager.LayoutParams.FLAG_SECURE)
    onDispose {
        window.clearFlags(WindowManager.LayoutParams.FLAG_SECURE)
    }
}
CreditCardDetails()
}

```

- Scrub sensitive state from ViewModel on app background (AppLifecycleObserver).
- Avoid logging sensitive data — strip logs in release via ProGuard: `-assumenosideeffects class android.util.Log { *; }`
- Use Android Keystore for cryptographic key storage — keys never leave the secure enclave.
- Validate all server-provided URLs before loading in WebView.
- Disable WebView JavaScript if not needed; never call `setJavaScriptEnabled(true)` unnecessarily.

7. Accessibility

Accessibility is a first-class concern in Compose. The Semantics system bridges your composables to Android's Accessibility Services (TalkBack, Switch Access, Voice Access). Well-structured semantics cost very little effort but dramatically expand your user base.

7.1 Semantics & Roles

Kotlin – Semantic roles and custom actions

```
@Composable
fun LikeButton(liked: Boolean, onToggle: () -> Unit) {
    val desc = if (liked) "Unlike post" else "Like post"
    IconButton(
        onClick = onToggle,
        modifier = Modifier.semantics {
            role = Role.Button
            contentDescription = desc
            stateDescription = if (liked) "Liked" else "Not liked"
        }
    ) {
        Icon(
            imageVector = if (liked) Icons.Filled.Favorite else Icons.Outlined.FavoriteBorder,
            contentDescription = null // handled by parent semantics
        )
    }
}
```

7.2 Content Descriptions

Kotlin – Meaningful and decorative images

```
// Informative image – MUST have description
Image(
    painter = painterResource(R.drawable.product_photo),
    contentDescription = "Red running shoes, front view"
)

// Decorative image – explicitly null to be skipped by TalkBack
Image(
    painter = painterResource(R.drawable.divider_pattern),
    contentDescription = null
)

// Merged semantics for a card
Card(modifier = Modifier.semantics(mergeDescendants = true) {}) {
    Row {
        Image(/* ... */)
        Column {
            Text("Product name")
            Text("$19.99")
        }
    }
}
```

```
}
```

7.3 Touch Target Size

Google and WCAG guidelines require a minimum touch target of 48x48dp. Use `Modifier.minimumInteractiveComponentSize()` (Compose 1.3+) or add padding manually.

Kotlin – Ensuring minimum touch target

```
@Composable
fun SmallIconButton(onClick: () -> Unit) {
    Box(
        modifier = Modifier
            .minimumInteractiveComponentSize() // enforces 48x48dp minimum
            .clickable(onClick = onClick),
        contentAlignment = Alignment.Center
    ) {
        Icon(Icons.Default.Close, contentDescription = "Close",
            modifier = Modifier.size(24.dp))
    }
}
```

7.4 TalkBack & Focus Management

Kotlin – Custom focus traversal and live regions

```
// Force focus to an element (e.g. error message after form submit)
val focusRequester = remember { FocusRequester() }
LaunchedEffect(hasError) {
    if (hasError) focusRequester.requestFocus()
}
Text(
    text = "Invalid email address",
    modifier = Modifier
        .focusRequester(focusRequester)
        .focusable()
        .semantics { liveRegion = LiveRegionMode.Polite }
)

// Exclude non-interactive decoration from accessibility tree
Box(modifier = Modifier.semantics { invisibleToUser() }) {
    DecorativeBackground()
}
```

- Test your app with TalkBack enabled on a real device — emulator TalkBack is unreliable.
- Use Accessibility Scanner from Google Play to detect common issues automatically.
- Ensure color contrast ratio is at least 4.5:1 for body text, 3:1 for large text.
- Do not convey information by color alone — add icons or text labels.
- Support font scaling: use sp for text sizes and test with largest font scale (200%).

8. Testing

8.1 Unit Testing ViewModels with StateFlow

Kotlin – ViewModel unit test

```
@OptIn(ExperimentalCoroutinesApi::class)
class HomeViewModelTest {

    @get:Rule val mainDispatcherRule = MainDispatcherRule()

    private val fakeRepo = FakeItemRepository()
    private lateinit var vm: HomeViewModel

    @Before fun setup() { vm = HomeViewModel(fakeRepo) }

    @Test fun `loading items emits success state`() = runTest {
        fakeRepo.items = listOf(Item(1, "Sample"))
        vm.loadItems()
        val state = vm.uiState.value
        assertThat(state.isLoading).isFalse()
        assertThat(state.items).hasSize(1)
    }
}
```

8.2 Compose UI Testing

Kotlin – ComposeTestRule interactions

```
class HomeScreenTest {

    @get:Rule val composeTestRule = createComposeRule()

    @Test fun `items are displayed`() {
        val items = listOf(Item(1, "Compose"), Item(2, "Android"))
        composeTestRule.setContent {
            MyAppTheme { ItemFeed(items) }
        }
        composeTestRule
            .onNodeWithText("Compose")
            .assertIsDisplayed()
        composeTestRule
            .onNodeWithContentDescription("Like post")
            .performClick()
            .assertIsSelected()
    }

    @Test fun `loading indicator shown while fetching`() {
        composeTestRule.setContent {
            MyAppTheme { HomeScreen() } // initial state isLoading=true
        }
        composeTestRule
            .onNodeWithTag("loading_indicator")
    }
}
```

```
.assertIsDisplayed()
}
}
```

8.3 Screenshot Testing

Kotlin – Paparazzi screenshot test

```
// build.gradle: testImplementation("app.cash.paparazzi:paparazzi:1.x")
class ProfileCardScreenshotTest {

    @get:Rule val paparazzi = Paparazzi(
        deviceConfig = DeviceConfig.PIXEL_5,
        theme = "android:Theme.Material.Light.NoActionBar"
    )

    @Test fun `profile card light mode`() {
        paparazzi.snapshot {
            MyAppTheme(darkTheme = false) {
                ProfileCard("Alex", "Senior Engineer")
            }
        }
    }
}
```

9. Dependency Injection with Hilt

Hilt is the recommended DI solution for Android. It integrates with Compose via `hiltViewModel()` and manages the lifecycle of injected `ViewModel` instances automatically.

Kotlin – Hilt setup

```
// 1. Annotate Application
@HiltAndroidApp
class MyApp : Application()

// 2. Annotate Activity / Fragment
@AndroidEntryPoint
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent { MyApp() }
    }
}

// 3. Define a Module
@Module
@InstallIn(SingletonComponent::class)
object NetworkModule {
    @Provides @Singleton
    fun provideRetrofit(): Retrofit =
        Retrofit.Builder()
            .baseUrl("https://api.example.com/")
            .addConverterFactory(MoshiConverterFactory.create())
            .build()

    @Provides @Singleton
    fun provideApiService(retrofit: Retrofit): ApiService =
        retrofit.create(ApiService::class.java)
}

// 4. Inject in ViewModel
@HiltViewModel
class DetailViewModel @Inject constructor(
    private val repo: ItemRepository,
    savedInstanceState: SavedStateHandle
) : ViewModel() {
    val itemId = savedInstanceState.get<Int>("itemId") ?: -1
}

// 5. Obtain in Composable
@Composable
fun DetailScreen(vm: DetailViewModel = hiltViewModel()) { ... }
```

10. Animations

10.1 animate*AsState — Simple Value Animations

Kotlin – animateDpAsState & animateColorAsState

```
@Composable
fun ExpandableCard(content: String) {
    var expanded by remember { mutableStateOf(false) }
    val cardHeight by animateDpAsState(if (expanded) 200.dp else 64.dp, label = "height")
    val bgColor by animateColorAsState(
        if (expanded) MaterialTheme.colorScheme.primaryContainer
        else MaterialTheme.colorScheme.surface,
        label = "color"
    )
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .height(cardHeight)
            .clickable { expanded = !expanded },
        colors = CardDefaults.cardColors(containerColor = bgColor)
    ) {
        if (expanded) Text(content, modifier = Modifier.padding(16.dp))
    }
}
```

10.2 AnimatedVisibility

Kotlin – Enter/exit transitions

```
@Composable
fun NotificationBanner(visible: Boolean, message: String) {
    AnimatedVisibility(
        visible = visible,
        enter = slideInVertically(initialOffsetY = { -it }) + fadeIn(),
        exit = slideOutVertically(targetOffsetY = { -it }) + fadeOut()
    ) {
        Surface(
            color = MaterialTheme.colorScheme.error,
            modifier = Modifier.fillMaxWidth()
        ) {
            Text(message, modifier = Modifier.padding(16.dp), color = Color.White)
        }
    }
}
```

10.3 Transition API — Multi-property Animations

Kotlin – updateTransition for complex state animations

```
enum class ButtonState { Idle, Loading, Success }

@Composable
fun AnimatedSubmitButton(state: ButtonState, onClick: () -> Unit) {
```

```

val transition = updateTransition(state, label = "button")

val width by transition.animateDp(label = "width") { s ->
when (s) { ButtonState.Idle -> 200.dp; else -> 56.dp }
}
val cornerRadius by transition.animateDp(label = "corner") { s ->
when (s) { ButtonState.Idle -> 8.dp; else -> 28.dp }
}

Button(
onClick = onClick,
modifier = Modifier.width(width),
shape = RoundedCornerShape(cornerRadius)
) {
when (state) {
ButtonState.Idle -> Text("Submit")
ButtonState.Loading -> CircularProgressIndicator(Modifier.size(20.dp))
ButtonState.Success -> Icon(Icons.Default.Check, contentDescription = "Done")
}
}
}

```

11. Networking & Data

11.1 Retrofit + Kotlin Flow

Kotlin – Repository pattern with Flow

```
// API Service
interface ApiService {
    @GET("items")
    suspend fun getItems(): Response<List<ItemDto>>
}

// Repository
class ItemRepositoryImpl @Inject constructor(
    private val api: ApiService
) : ItemRepository {
    override fun getItems(): Flow<Result<List<Item>>> = flow {
        try {
            val response = api.getItems()
            if (response.isSuccessful) {
                emit(Result.success(response.body()!!.map { it.toDomain() })))
            } else {
                emit(Result.failure(HttpException(response)))
            }
        } catch (e: IOException) {
            emit(Result.failure(e))
        }
    }.flowOn(Dispatchers.IO)
}
```

11.2 Room Database

Kotlin – Room with Flow DAO

```
@Entity(tableName = "items")
data class ItemEntity(
    @PrimaryKey val id: Int,
    val name: String,
    val cachedAt: Long = System.currentTimeMillis()
)

@Dao
interface ItemDao {
    @Query("SELECT * FROM items ORDER BY name ASC")
    fun observeItems(): Flow<List<ItemEntity>> // reactive – emits on every change

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertAll(items: List<ItemEntity>)

    @Query("DELETE FROM items")
    suspend fun clearAll()
}
```

```
@Database(entities = [ItemEntity::class], version = 1, exportSchema = true)
abstract class AppDatabase : RoomDatabase() {
    abstract fun itemDao(): ItemDao
}
```

11.3 DataStore (Preferences)

Kotlin – DataStore for typed user preferences

```
val Context.dataStore: DataStore<Preferences> by preferencesDataStore(name = "settings")

object PreferencesKeys {
    val DARK_MODE = booleanPreferencesKey("dark_mode")
    val FONT_SCALE = floatPreferencesKey("font_scale")
}

class SettingsRepository(private val context: Context) {

    val isDarkMode: Flow<Boolean> = context.dataStore.data
        .map { prefs -> prefs[PreferencesKeys.DARK_MODE] ?: false }

    suspend fun setDarkMode(enabled: Boolean) {
        context.dataStore.edit { prefs ->
            prefs[PreferencesKeys.DARK_MODE] = enabled
        }
    }
}
```

12. Coding Standards

Consistent coding standards reduce cognitive overhead, simplify code reviews, and prevent entire categories of bugs. Android/Kotlin development has well-established conventions endorsed by JetBrains, Google, and the broader community.

12.1 Kotlin Naming Conventions

Element	Convention	Example
Class / Object	PascalCase	UserProfileViewModel
Interface	PascalCase	ItemRepository
Function / Method	camelCase	fetchUserData()
Variable / Property	camelCase	isLoading, userName
Constant (companion)	SCREAMING_SNAKE	MAX_RETRY_COUNT
Package	lowercase, no underscores	com.example.app.feature
File	PascalCase = class name	HomeScreen.kt
Enum entry	PascalCase	LoadState.Success
Type parameter	Single upper letter	T, K, V, R

Kotlin – Naming in practice

```
// Classes & Interfaces
interface UserRepository { ... }
class UserRepositoryImpl : UserRepository { ... }

// Constants in companion object
class NetworkConfig {
    companion object {
        const val BASE_URL = "https://api.example.com/"
        const val TIMEOUT_SECS = 30L
        const val MAX_RETRY = 3
    }
}

// Boolean properties: is/has/should prefix
val isAuthenticated: Boolean
val hasUnreadMessages: Boolean
val shouldShowOnboarding: Boolean

// Backing properties: underscore prefix
private val _uiState = MutableStateFlow(HomeUiState())
val uiState: StateFlow<HomeUiState> = _uiState.asStateFlow()
```

12.2 Composable Naming Rules

Composable function names follow PascalCase (unlike regular functions) because they represent UI components — conceptual nouns, not verbs. This is enforced by Compose's own linting rules.

Kotlin – Composable naming rules

```
// GOOD: PascalCase, noun describing the UI element
@Composable fun UserProfileCard(user: User) { ... }
@Composable fun LoadingIndicator() { ... }
@Composable fun SearchBar(query: String, onQueryChange: (String) -> Unit) { ... }

// BAD: camelCase (looks like a regular function)
// @Composable fun userProfileCard(user: User) { ... }

// GOOD: Composable that returns a value uses camelCase (not a UI node)
@Composable fun rememberScrollState(): ScrollState = remember { ScrollState(0) }

// Composable lambdas in parameters: 'content' or descriptive noun
@Composable
fun AppScaffold(
    topBar: @Composable () -> Unit = {},
    content: @Composable (PaddingValues) -> Unit
) { ... }
```

12.3 File & Package Structure

Feature-first package structure (also called vertical slicing) scales better than layer-first as the project grows. Each feature is self-contained and can be extracted into a module independently.

Directory – Recommended project layout

```
app/
src/main/
java/com/example/app/
core/
data/ (network, database, repositories)
domain/ (models, use cases, repository interfaces)
ui/
components/ (shared reusable composables)
theme/ (Color.kt, Type.kt, Theme.kt, Shape.kt)
feature/
home/
data/ HomeRepository.kt, HomeApiService.kt
domain/ HomeUseCase.kt, HomeModel.kt
ui/ HomeScreen.kt, HomeViewModel.kt,
HomeUiState.kt, HomeEvent.kt
profile/
...
auth/
...
di/ AppModule.kt, NetworkModule.kt
MainActivity.kt
MyApp.kt
```

12.4 Modifier Convention

Always accept a Modifier parameter as the first (or only) named parameter after required data parameters, defaulting to Modifier. This pattern lets callers customize layout without breaking the component's internal logic.

Kotlin – Modifier parameter convention

```
// CORRECT: modifier is first optional param, defaults to Modifier
@Composable
fun AvatarImage(
    imageUrl: String, // required data first
    modifier: Modifier = Modifier, // modifier second, with default
    size: Dp = 40.dp // other optional config after
) {
    AsyncImage(
        model = imageUrl,
        contentDescription = null,
        modifier = modifier // pass through directly – never wrap unless needed
    ).size(size)
    .clip(CircleShape)
}

// WRONG: modifier not accepted (forces callers to wrap in Box)
// @Composable fun BadAvatar(url: String) { AsyncImage(model = url, ...) }

// Chaining order: sizing -> padding -> background -> clickable -> semantics
Modifier
    .fillMaxWidth()
    .height(56.dp)
    .padding(horizontal = 16.dp)
    .background(MaterialTheme.colorScheme.surface)
    .clickable(onClick = onClick)
    .semantics { role = Role.Button }
```

12.5 Parameter Ordering in Composables

Consistent parameter order makes composables predictable and aids auto-complete. Follow this canonical ordering:

- 1. Required parameters (no default) — most important data first.
- 2. modifier: Modifier = Modifier — always named 'modifier'.
- 3. Optional data parameters with defaults.
- 4. Event callbacks (lambdas) — last, named 'onX' for actions.
- 5. content: @Composable () -> Unit — very last if present.

Kotlin – Parameter ordering example

```
@Composable
fun ProductCard(
    // 1. Required data
    title: String,
    price: Double,
    imageUrl: String,
    // 2. Modifier
    modifier: Modifier = Modifier,
    // 3. Optional config
```

```

isOnSale: Boolean = false,
badgeText: String? = null,
// 4. Event callbacks
onAddToCart: () -> Unit,
onFavourite: () -> Unit = {},
// 5. Slot content
footer: @Composable (() -> Unit)? = null
) { ... }

```

12.6 Preview Annotations

Write `@Preview` functions for every composable. Provide multiple previews covering light/dark, font scales, and device sizes. Keep previews in the same file as the composable.

Kotlin – Comprehensive preview setup

```

// Define reusable preview annotations
@Preview(name = "Light", uiMode = UI_MODE_NIGHT_NO, showBackground = true)
@Preview(name = "Dark", uiMode = UI_MODE_NIGHT_YES, showBackground = true)
annotation class ThemePreview

@Preview(name = "Small font", fontScale = 0.85f)
@Preview(name = "Large font", fontScale = 1.5f)
annotation class FontScalePreview

// Apply to composables
@ThemePreview
@FontScalePreview
@Composable
private fun ProductCardPreview() {
    MyAppTheme {
        ProductCard(
            title = "Wireless Headphones",
            price = 49.99,
            imageUrl = "https://example.com/img.jpg",
            isOnSale = true,
            onAddToCart = {},
            onFavourite = {}
        )
    }
}

```

12.7 Code Formatting & Linting (ktlint / detekt)

Groovy – Gradle setup for ktlint

```

// settings.gradle.kts – add plugin
plugins {
    id("org.jlleitschuh.gradle.ktlint") version "12.x.x"
}

// build.gradle.kts (app or root)
ktlint {
    android.set(true)
    outputToConsole.set(true)
}

```

```
ignoreFailures.set(false)
reporters {
    reporter(ReporterType.PLAIN)
    reporter(ReporterType.CHECKSTYLE)
}
filter {
    exclude("**/generated/**")
}
}
```

Shell – Run ktlint / detekt

```
# Check formatting
./gradlew ktlintCheck

# Auto-format
./gradlew ktlintFormat

# Static analysis with detekt
./gradlew detekt
```

YAML – .editorconfig for ktlint rules

```
[*.{kt,kts}]
indent_size = 4
max_line_length = 120
ktlint_standard_no-wildcard-imports = enabled
ktlint_compose_composable-naming = enabled
ktlint_compose_modifier-missing = enabled
ktlint_compose_preview-naming = enabled
```

13. Kotlin Best Practices

13.1 Null Safety

Kotlin's null safety system eliminates `NullPointerException`s at compile time. Embrace it — avoid `!!` force-unwrapping except in verified-safe contexts.

Kotlin – Null safety patterns

```
// BAD: force unwrap – crashes if null
val name = user!!.name

// GOOD: safe call + elvis operator
val name = user?.name ?: "Anonymous"

// GOOD: let for null-guarded blocks
user?.let { u ->
    showProfile(u.name, u.avatar)
}

// GOOD: requireNotNull for contract enforcement
fun processUser(user: User?) {
    val validUser = requireNotNull(user) { "User must not be null at this point" }
    // validUser is non-null here
}

// GOOD: early return (guard clause) pattern
fun renderUser(user: User?) {
    user ?: return // nothing to render
    Text(user.name)
}
```

13.2 Scope Functions

Function	Context	Returns	Best Use Case
let	it	lambda	Null-safety blocks, transform & chain
run	this	lambda	Object config + compute result
with	this	lambda	Grouping calls on an object (non-extension)
apply	this	receiver	Object initialisation / builder pattern
also	it	receiver	Side effects (logging, validation)

Kotlin – Scope function usage

```
// apply – object initialisation
val paint = Paint().apply {
    color = Color.RED
    strokeWidth = 4f
    isAntiAlias = true
}
```

```
// let – null-safe transform
val upperName = user?.name?.let { it.uppercase().trim() }

// also – side effect without breaking chain
return fetchData()
.also { data -> logger.log("Fetched ${data.size} items") }

// run – compute with context
val summary = user.run {
    "${name.uppercase()} (${email})"
}
```

13.3 Extension Functions

Kotlin – Useful extension patterns

```
// String extensions
fun String.isValidEmail(): Boolean =
    android.util.Patterns.EMAIL_ADDRESS.matcher(this).matches()

fun String.toTitleCase(): String =
    split(" ").joinToString(" ") { it.replaceFirstChar(Char::uppercase) }

// Context extensions
fun Context.showToast(msg: String, duration: Int = Toast.LENGTH_SHORT) =
    Toast.makeText(this, msg, duration).show()

// Flow extensions
fun <T> Flow<T>.throttleFirst(windowDuration: Long): Flow<T> = flow {
    var lastEmit = 0L
    collect { value ->
        val now = System.currentTimeMillis()
        if (now - lastEmit >= windowDuration) {
            lastEmit = now
            emit(value)
        }
    }
}

// Modifier extensions
fun Modifier.visible(visible: Boolean): Modifier =
    if (visible) this else this.then(Modifier.alpha(0f).size(0.dp))
```

13.4 Sealed Classes & When Expressions

Kotlin – Exhaustive when with sealed classes

```
sealed class NetworkResult<out T> {
    data class Success<T>(val data: T) : NetworkResult<T>()
    data class Error(val code: Int, val message: String) : NetworkResult<Nothing>()
    object Loading : NetworkResult<Nothing>()
    object Empty : NetworkResult<Nothing>()
}
```

```

// Exhaustive when (compiler enforces all branches)
fun <T> NetworkResult<T>.handle(
    onSuccess: (T) -> Unit,
    onError: (Int, String) -> Unit,
    onLoading: () -> Unit = {},
    onEmpty: () -> Unit = {}
) = when (this) {
    is NetworkResult.Success -> onSuccess(data)
    is NetworkResult.Error -> onError(code, message)
    NetworkResult.Loading -> onLoading()
    NetworkResult.Empty -> onEmpty()
}

// In Compose
when (val result = uiState.result) {
    is NetworkResult.Success -> ItemList(result.data)
    is NetworkResult.Error -> ErrorMessage(result.message)
    NetworkResult.Loading -> LoadingSpinner()
    NetworkResult.Empty -> EmptyState()
}

```

13.5 Coroutines & Flow Patterns

Kotlin – Flow operators and structured concurrency

```

// Combine multiple flows
combine(
    repo.getUser(),
    repo.getNotifications()
) { user, notifications ->
    HomeUiState(user = user, badgeCount = notifications.size)
}
.stateIn(
    scope = viewModelScope,
    started = SharingStarted.WhileSubscribed(5_000),
    initialValue = HomeUiState()
)

// Retry with exponential backoff
repo.getItems()
.retry(3) { e -> e is IOException }
.catch { e -> _uiState.update { it.copy(error = e.message) } }
.collect { items -> _uiState.update { it.copy(items = items) } }

// Debounce search input
searchQuery
.debounce(300)
.filter { it.length >= 2 }
.distinctUntilChanged()
.flatMapLatest { query -> repo.search(query) }
.collect { results -> _uiState.update { it.copy(results = results) } }

```

13.6 Data Classes & Value Classes

Kotlin – Value classes to prevent primitive obsession

```
// Wrap primitives to prevent mix-ups
@JvmInline
value class UserId(val value: Int)

@JvmInline
value class Email(val value: String) {
    init { require(value.contains("@")) { "Invalid email: $value" } }
}

// Usage – compiler prevents passing wrong type
fun getUser(id: UserId): User = ...
// getUser(42) // compile error
// getUser(UserId(42)) // correct

// Data class: always override equals/hashCode, provide copy()
data class User(
    val id: UserId,
    val name: String,
    val email: Email
)
val updated = user.copy(name = "New Name")
```

14. Error Handling & Logging

14.1 Result / Either Pattern

Use Kotlin's built-in Result or a custom Either type to represent success/failure without exceptions crossing architectural boundaries. Exceptions should be caught at the data layer and converted to typed errors.

Kotlin – Custom Result wrapper

```
sealed class AppError {
    data class Network(val code: Int, val msg: String) : AppError()
    data class Validation(val field: String, val reason: String) : AppError()
    object Unauthorized : AppError()
    object Unknown : AppError()
}

typealias AppResult<T> = Result<T> // or use Arrow Either

// Repository converts exceptions to typed errors
suspend fun fetchUser(id: UserId): AppResult<User> =
    runCatching {
        val dto = api.getUser(id.value)
        dto.toDomain()
    }.mapFailure { throwable ->
        when (throwable) {
            is HttpException -> AppError.Network(throwable.code(), throwable.message())
            is IOException -> AppError.Network(-1, "No internet connection")
            else -> AppError.Unknown
        }
    }

// ViewModel handles typed errors
result.onSuccess { user ->
    _uiState.update { it.copy(user = user) }
}.onFailure { error ->
    val msg = when (error) {
        is AppError.Network -> "Connection error: ${error.msg}"
        AppError.Unauthorized -> "Please log in again"
        else -> "Something went wrong"
    }
    _uiState.update { it.copy(errorMessage = msg) }
}
```

14.2 Global Error Handling

Kotlin – CoroutineExceptionHandler + Compose SnackbarHost

```
// ViewModel-level exception handler
val handler = CoroutineExceptionHandler { _, throwable ->
    _uiState.update { it.copy(error = throwable.message) }
}

// Launch with handler
viewModelScope.launch(handler) {
```



```

riskyOperation()
}

// Composable – show errors via Snackbar
@Composable
fun HomeScreen(vm: HomeViewModel = hiltViewModel()) {
    val snackbarHostState = remember { SnackbarHostState() }
    val state by vm.uiState.collectAsStateWithLifecycle()

    LaunchedEffect(state.error) {
        state.error?.let { msg ->
            snackbarHostState.showSnackbar(msg)
            vm.clearError()
        }
    }

    Scaffold(snackbarHost = { SnackbarHost(snackbarHostState) }) { padding ->
        Content(Modifier.padding(padding))
    }
}

```

14.3 Structured Logging

Kotlin – Timber logging setup

```

// build.gradle: implementation("com.jakewharton.timber:timber:5.x")

// Application class
class MyApp : Application() {
    override fun onCreate() {
        super.onCreate()
        if (BuildConfig.DEBUG) {
            Timber.plant(Timber.DebugTree())
        } else {
            Timber.plant(CrashReportingTree()) // sends to Crashlytics
        }
    }
}

private class CrashReportingTree : Timber.Tree() {
    override fun log(priority: Int, tag: String?, message: String, t: Throwable?) {
        if (priority >= Log.WARN) {
            FirebaseCrashlytics.getInstance().log(message)
            t?.let { FirebaseCrashlytics.getInstance().recordException(it) }
        }
    }
}

// Usage
Timber.d("User loaded: %s", userId)
Timber.w("Slow response: %d ms", responseTime)
Timber.e(exception, "Failed to load items")

```

- Never log PII (email, name, tokens) even in debug builds.

- Use `Timber.tag()` to group logs by feature for easier filtering.
- Strip all `Timber.d` calls in release using ProGuard: `-assumenosideeffects class timber.log.Timber { v(...); d(...); i(...); }`

14.4 Crash Reporting (Firebase Crashlytics)

Kotlin – Crashlytics custom keys & non-fatal exceptions

```
// Record breadcrumbs for better crash context
FirebaseCrashlytics.getInstance().apply {
    setCustomKey("user_id", userId.value)
    setCustomKey("screen", "HomeScreen")
    setCustomKey("build_type", BuildConfig.BUILD_TYPE)
    log("User tapped refresh button")
}

// Non-fatal exceptions (track without crashing)
try {
    parseComplexData(rawJson)
} catch (e: JsonParseException) {
    FirebaseCrashlytics.getInstance().recordException(e)
} // fallback behavior
```

15. Gradle & Build Best Practices

15.1 Version Catalogs (libs.versions.toml)

Version catalogs are the modern replacement for scattered `ext{}` variables. They provide type-safe dependency accessors, a single source of truth for versions, and IDE auto-complete.

TOML – gradle/libs.versions.toml

```
[versions]
kotlin = "2.0.0"
compose-bom = "2024.09.00"
hilt = "2.52"
room = "2.6.1"
retrofit = "2.11.0"
coil = "2.7.0"
coroutines = "1.8.1"

[libraries]
compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "compose-bom" }
compose-ui = { group = "androidx.compose.ui", name = "ui" }
compose-material3 = { group = "androidx.compose.material3", name = "material3" }
hilt-android = { module = "com.google.dagger:hilt-android", version.ref = "hilt" }
hilt-compiler = { module = "com.google.dagger:hilt-android-compiler", version.ref = "hilt" }
room-runtime = { module = "androidx.room:room-runtime", version.ref = "room" }
room-compiler = { module = "androidx.room:room-compiler", version.ref = "room" }
room-ktx = { module = "androidx.room:room-ktx", version.ref = "room" }

[plugins]
android-application = { id = "com.android.application", version = "8.5.0" }
kotlin-android = { id = "org.jetbrains.kotlin.android", version.ref = "kotlin" }
hilt = { id = "com.google.dagger.hilt.android", version.ref = "hilt" }
ksp = { id = "com.google.devtools.ksp", version = "2.0.0-1.0.24" }
```

Kotlin DSL – build.gradle.kts usage

```
dependencies {
    implementation(platform(libs.compose.bom))
    implementation(libs.compose.ui)
    implementation(libs.compose.material3)
    implementation(libs.hilt.android)
    ksp(libs.hilt.compiler)
    implementation(libs.room.runtime)
    implementation(libs.room.ktx)
    ksp(libs.room.compiler)
}
```

15.2 Build Variants & Flavors

Kotlin DSL – Build types & product flavors

```
android {
    buildTypes {
```

```

debug {
    applicationIdSuffix = ".debug"
    versionNameSuffix = "-DEBUG"
    isDebuggable = true
    isMinifyEnabled = false
}
release {
    isMinifyEnabled = true
    isShrinkResources = true
    signingConfig = signingConfigs.getByName("release")
}
create("staging") {
    initWith(getByName("release"))
    applicationIdSuffix = ".staging"
    isDebuggable = true
}
}

flavorDimensions += "environment"
productFlavors {
    create("dev") { dimension = "environment";
        buildConfigField("String", "BASE_URL", "'https://dev.api.com/'") }
    create("prod") { dimension = "environment";
        buildConfigField("String", "BASE_URL", "'https://api.example.com/'") }
}
}

```

15.3 Multi-Module Architecture

Directory – Module layout

```

root/
app/ (thin shell – wires features together)
core/
:core:network (Retrofit, OkHttp, interceptors)
:core:database (Room setup, base DAOs)
:core:ui (shared composables, theme)
:core:domain (base use-case, AppError, Result)
:core:testing (test utilities, fake repositories)
feature/
:feature:home (depends on :core:domain, :core:ui)
:feature:profile
:feature:auth

```

- Keep :app module minimal — only navigation graph and DI wiring.
- Feature modules must NOT depend on each other; go through :core:domain.
- Use `api()` vs `implementation()` in dependencies to control transitive leakage.
- Enable `buildFeatures { buildConfig = true }` only in modules that need it.

15.4 Build Performance

Properties – gradle.properties optimisations

```

# Increase heap for large projects
org.gradle.jvmargs=-Xmx4g -XX:+UseParallelGC

```

```
# Parallel builds across modules
org.gradle.parallel=true

# Cache build outputs
org.gradle.caching=true

# Configuration cache (Gradle 8+)
org.gradle.configuration-cache=true

# Kotlin incremental compilation
kotlin.incremental=true
kotlin.incremental.useClasspathSnapshot=true

# Compose compiler metrics (enable when profiling only)
# compose.generateCompilerMetrics=true
# compose.generateCompilerReports=true
```

16. CI/CD for Android

16.1 GitHub Actions Workflow

YAML – `.github/workflows/android-ci.yml`

```
name: Android CI

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'

      - name: Cache Gradle
        uses: actions/cache@v4
        with:
          path: |
            ~/.gradle/caches
            ~/.gradle/wrapper
          key: ${{ runner.os }}-gradle-${{ hashFiles('**/*.gradle*', '**/libs.versions.toml') }}

      - name: Grant execute permission
        run: chmod +x gradlew

      - name: Run ktlint
        run: ./gradlew ktlintCheck

      - name: Run detekt
        run: ./gradlew detekt

      - name: Run unit tests
        run: ./gradlew testDebugUnitTest

      - name: Build release APK
        run: ./gradlew assembleRelease

      - name: Upload test results
        uses: actions/upload-artifact@v4
        if: always()
```

```
with:
name: test-results
path: '**/build/reports/tests/'
```

16.2 Static Analysis

Kotlin DSL – detekt configuration

```
// build.gradle.kts
plugins {
id("io.gitlab.arturbosch.detekt") version "1.x"
}

detekt {
config.setFrom("$projectDir/config/detekt.yml")
buildUponDefaultConfig = true
parallel = true
}

// config/detekt.yml
complexity:
LongMethod:
threshold: 40 // max lines per function
LongParameterList:
functionThreshold: 6 // max params

naming:
FunctionNaming:
functionPattern: '[a-z][a-zA-Z0-9]*' // camelCase functions
ignoreAnnotated: ['Composable'] // allow PascalCase for @Composable
```

16.3 Automated Testing in CI

YAML – Run instrumented tests on emulator

```
instrumented-tests:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- uses: actions/setup-java@v4
with: { java-version: '17', distribution: 'temurin' }

- name: Enable KVM
run: |
echo 'KERNEL=="kvm", GROUP="kvm", MODE="0666" | sudo tee
/etc/udev/rules.d/99-kvm4all.rules
sudo udevadm control --reload-rules
sudo udevadm trigger --name-match=kvm

- name: Run instrumented tests
uses: reactivecircus/android-emulator-runner@v2
with:
api-level: 34
target: google_apis
arch: x86_64
```

```
script: ./gradlew connectedDebugAndroidTest

- name: Upload test reports
  uses: actions/upload-artifact@v4
  with:
    name: instrumented-test-results
    path: '**/build/reports/androidTests/'
```

16.4 Play Store Deployment

YAML – Deploy to Play Store via Fastlane

```
deploy:
  runs-on: ubuntu-latest
  needs: build-and-test
  if: github.ref == 'refs/heads/main'
  steps:
    - uses: actions/checkout@v4
    - name: Deploy to Play Store (Internal Track)
      uses: r0adkll/upload-google-play@v1
      with:
        serviceAccountJsonPlainText: ${ secrets.SERVICE_ACCOUNT_JSON }
        packageName: com.example.app
        releaseFiles: app/build/outputs/bundle/release/*.aab
        track: internal
        status: completed
        mappingFile: app/build/outputs/mapping/release/mapping.txt
```

- Always use Android App Bundles (.aab) over APKs for Play Store uploads.
- Store signing keystore and credentials in GitHub Secrets — never commit to repo.
- Enable Play App Signing; only upload the upload key to CI.
- Tag releases in git (v1.2.3) and derive versionCode from BUILD_NUMBER in CI.
- Use Play Store's internal testing track for CI deployments before promoting to production.

17. Battery Life Optimization

Battery drain is one of the top reasons users uninstall apps. Android provides a suite of APIs — `WorkManager`, `JobScheduler`, Doze mode, and batched network calls — that let you defer and batch background work so it runs efficiently when the device is already awake, charging, or connected to Wi-Fi.

17.1 WorkManager for Deferrable Background Work

`WorkManager` is the recommended API for any background work that must be guaranteed to execute, even if the app exits or the device restarts. It automatically selects the best underlying job-scheduling mechanism (`JobScheduler` / `AlarmManager`).

Kotlin — `WorkManager` periodic sync

```
// Define the Worker
class SyncWorker(ctx: Context, params: WorkerParameters) : CoroutineWorker(ctx, params) {
    override suspend fun doWork(): Result {
        return try {
            syncRepository.syncAll()
            Result.success()
        } catch (e: Exception) {
            if (runAttemptCount < 3) Result.retry() else Result.failure()
        }
    }
}

// Schedule it — runs at most once every 6 hours, only on Wi-Fi + charging
val constraints = Constraints.Builder()
    .setRequiredNetworkType(NetworkType.UNMETERED) // Wi-Fi only
    .setRequiresCharging(true) // charging only
    .setRequiresBatteryNotLow(true) // skip if battery low
    .build()

val syncRequest = PeriodicWorkRequestBuilder<SyncWorker>(6, TimeUnit.HOURS)
    .setConstraints(constraints)
    .setBackoffCriteria(BackoffPolicy.EXPONENTIAL, 15, TimeUnit.MINUTES)
    .addTag("sync")
    .build()

WorkManager.getInstance(context).enqueueUniquePeriodicWork(
    "periodic_sync",
    ExistingPeriodicWorkPolicy.KEEP,
    syncRequest
)
```

Kotlin — One-time expedited work (user-visible task)

```
class UploadWorker(ctx: Context, params: WorkerParameters) : CoroutineWorker(ctx, params) {
    override suspend fun getForegroundInfo(): ForegroundInfo =
        ForegroundInfo(NOTIF_ID, buildUploadNotification(applicationContext))

    override suspend fun doWork(): Result {
        uploadService.upload(inputData.getString("file_path") ?: return Result.failure())
        return Result.success()
    }
}
```

```

}
}

val request = OneTimeWorkRequestBuilder<UploadWorker>()
    .setExpedited(OutOfQuotaPolicy.RUN_AS_NON_EXPEDITED_WORK_REQUEST)
    .setInputData(workDataOf("file_path" to filePath))
    .build()

WorkManager.getInstance(context).enqueue(request)

```

17.2 Doze Mode & App Standby Buckets

Android 6+ automatically restricts network access and wafers CPU wake locks when the device is idle (Doze). Android 9+ classifies apps into standby buckets that further limit job and alarm frequency. Design your app assuming these restrictions will apply at any time.

Restriction	Doze Mode Behaviour	Workaround
Network access	Blocked during idle windows	Use maintenance windows; WorkManager handles automa
Wake locks	Ignored	WorkManager + Constraints; never roll your own
AlarmManager	Deferred until next maintenance window	Use setAndAllowWhileIdle() only for critical alarms
FCM / Push	High-priority FCM messages bypass Doze	Use FCM for time-sensitive user notifications
Standby bucket	Active > Working set > Frequent > Rare	Encourage user engagement; request exemption only for h

Kotlin – Check and request battery exemption (power-critical apps only)

```

val pm = getSystemService(PowerManager::class.java)
if (!pm.isIgnoringBatteryOptimizations(packageName)) {
    // Only for apps with genuine always-on requirement (e.g. medical, finance)
    val intent = Intent(Settings.ACTION_REQUEST_IGNORE_BATTERY_OPTIMIZATIONS)
    .setData(Uri.parse("package:$packageName"))
    startActivity(intent)
}

```

- Do NOT request battery optimization exemption unless absolutely justified — Google Play policy may reject your app.
- Test Doze behaviour: adb shell dumpsys deviceidle force-idle
- Test standby buckets: adb shell am set-standby-bucket com.example.app rare
- Use adb shell dumpsys battery unplug to simulate unplugged state during testing.

17.3 Foreground Services

Foreground services are for tasks that are user-visible and ongoing, such as music playback, navigation, or fitness tracking. Android 14+ requires you to declare the specific foreground service type and request the matching permission.

Kotlin – Foreground service (Android 14+)

```

// AndroidManifest.xml
// <uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
// <uses-permission android:name="android.permission.FOREGROUND_SERVICE_MEDIA_PLAYBACK" />
// <service android:name=".PlaybackService"

```

```
// android:foregroundServiceType="mediaPlayback" />

class PlaybackService : Service() {
    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
        val notification = buildPlayerNotification()
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
            startForeground(NOTIF_ID, notification,
                ServiceInfo.FOREGROUND_SERVICE_TYPE_MEDIA_PLAYBACK)
        } else {
            startForeground(NOTIF_ID, notification)
        }
        return START_STICKY
    }
    override fun onBind(intent: Intent?): IBinder? = null
}
```

17.4 Efficient Wake Locks & Alarms

Kotlin – Exact alarm for critical time-sensitive reminders

```
val alarmManager = getSystemService(AlarmManager::class.java)

// Android 12+: must check permission before scheduling exact alarms
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
    if (!alarmManager.canScheduleExactAlarms()) {
        startActivity(Intent(Settings.ACTION_REQUEST_SCHEDULE_EXACT_ALARM))
        return
    }
}

val pendingIntent = PendingIntent.getBroadcast(
    context, REQUEST_CODE, Intent(context, AlarmReceiver::class.java),
    PendingIntent.FLAG_IMMUTABLE or PendingIntent.FLAG_UPDATE_CURRENT
)

// Use setExactAndAllowWhileIdle ONLY for user-initiated reminders
alarmManager.setExactAndAllowWhileIdle(
    AlarmManager.RTC_WAKEUP,
    triggerTimeMillis,
    pendingIntent
)
```

- Prefer inexact alarms (`setInexactRepeating`) for non-critical periodic tasks — OS batches them for efficiency.
- Never use `PowerManager.WakeLock` directly; `WorkManager` manages this for you.
- Always release `WakeLocks` in finally blocks if you must use them.
- Use `AlarmManager.ELAPSED_REALTIME` (not `RTC_WAKEUP`) when you don't need to wake the device.

17.5 Network & Sensor Batching

Kotlin – Batch network requests with `OkHttp` connection pooling

```
// Single shared OkHttpClient – reuses TCP connections
val okHttpClient = OkHttpClient.Builder()
```

```

.connectionPool(ConnectionPool(5, 5, TimeUnit.MINUTES))
.connectTimeout(10, TimeUnit.SECONDS)
.readTimeout(30, TimeUnit.SECONDS)
// Compress requests to reduce data and battery
.addInterceptor(GzipRequestInterceptor())
.build()

// Use conditional GET with ETags to avoid re-downloading unchanged data
.cacheControl(CacheControl.Builder().maxAge(1, TimeUnit.HOURS).build())

```

Kotlin – Batched sensor reading (step counter)

```

val sensorManager = getSystemService(SensorManager::class.java)
val stepSensor = sensorManager.getDefaultSensor(Sensor.TYPE_STEP_COUNTER)

// SENSOR_DELAY_NORMAL batches readings – much more battery efficient than FASTEST
sensorManager.registerListener(
    stepListener,
    stepSensor,
    SensorManager.SENSOR_DELAY_NORMAL,
    maxReportLatencyUs = 5_000_000 // batch up to 5 seconds of data
)

```

18. Secure Communication

Secure communication protects data in transit from eavesdropping, tampering, and man-in-the-middle attacks. All Android apps must use HTTPS for every network call and should layer additional defences such as certificate pinning, token management, and payload encryption where appropriate.

18.1 TLS / HTTPS Enforcement

XML – Network Security Config: enforce HTTPS everywhere

```
<!-- res/xml/network_security_config.xml -->
<network-security-config>
  <base-config cleartextTrafficPermitted="false">
    <trust-anchors>
      <certificates src="system" />
    </trust-anchors>
  </base-config>
</network-security-config>

<!-- AndroidManifest.xml -->
<application
  android:networkSecurityConfig="@xml/network_security_config"
  .../>
```

Kotlin – OkHttp TLS minimum version

```
val tlsSpec = ConnectionSpec.Builder(ConnectionSpec.MODERN_TLS)
    .tlsVersions(TlsVersion.TLS_1_2, TlsVersion.TLS_1_3)
    .cipherSuites(
        CipherSuite.TLS_AES_128_GCM_SHA256,
        CipherSuite.TLS_AES_256_GCM_SHA384,
        CipherSuite.TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
    )
    .build()

val client = OkHttpClient.Builder()
    .connectionSpecs(listOf(tlsSpec))
    .build()
```

18.2 Certificate Pinning

Certificate pinning rejects TLS certificates that are not in your explicit allowlist, preventing MITM attacks even if a CA is compromised. Always pin at least two certificates (primary + backup) and set a renewal schedule.

Kotlin – OkHttp CertificatePinner

```
// Get pin: openssl s_client -connect api.example.com:443 | openssl x509 -pubkey -noout |
// openssl rsa -pubin -outform der | openssl dgst -sha256 -binary | base64
val pinner = CertificatePinner.Builder()
    .add("api.example.com", "sha256/primaryPinBase64==")
    .add("api.example.com", "sha256/backupPinBase64==") // always include a backup
    .build()

val client = OkHttpClient.Builder()
    .certificatePinner(pinner)
```

```

.build()

// Handle pin mismatch (certificate replaced by attacker or expired rotation)
try {
    client.newCall(request).execute()
} catch (e: SSLPeerUnverifiedException) {
    // Log + alert + fall back to graceful error UI
    Timber.e(e, "Certificate pin mismatch – possible MITM attack")
}

```

- Set an expiry date for pins and automate rotation reminders.
- Use `network_security_config` as an alternative to OkHttp pinner.
- Never ship debug builds with pinning disabled in production.
- During development, use a Charles/mitmproxy CA ONLY in debug build `networkSecurityConfig`.

18.3 OAuth 2.0 & Token Management

Kotlin – AppAuth for OAuth 2.0 PKCE flow

```

// build.gradle: implementation("net.openid:appauth:0.11.1")

// Build the authorization request with PKCE
val serviceConfig = AuthorizationServiceConfiguration(
    Uri.parse("https://auth.example.com/authorize"),
    Uri.parse("https://auth.example.com/token")
)
val authRequest = AuthorizationRequest.Builder(
    serviceConfig,
    CLIENT_ID,
    ResponseTypeValues.CODE,
    Uri.parse("com.example.app:/callback")
)
    .setScope("openid profile email")
    .build()

// Launch browser-based auth (PKCE generated automatically)
val authService = AuthorizationService(context)
val intent = authService.getAuthorizationRequestIntent(authRequest)
authLauncher.launch(intent)

// Exchange code for tokens (in ViewModel)
authService.performTokenRequest(tokenRequest) { response, ex ->
    if (response != null) {
        // Store tokens encrypted – never in plain SharedPreferences
        secureStorage.saveTokens(
            accessToken = response.accessToken,
            refreshToken = response.refreshToken,
            expiry = response.accessTokenExpirationTime
        )
    }
}

```

Kotlin – Token refresh interceptor (OkHttp)

```

class AuthInterceptor(private val tokenManager: TokenManager) : Interceptor {
    override fun intercept(chain: Interceptor.Chain): Response {
        val token = tokenManager.getValidAccessToken() // refreshes if expired
        val request = chain.request().newBuilder()
            .header("Authorization", "Bearer $token")
            .build()
        val response = chain.proceed(request)
        if (response.code == 401) {
            tokenManager.clearTokens()
            // Navigate to login via event bus / SharedFlow
        }
        return response
    }
}

```

18.4 Encrypted Payloads

Kotlin – AES-GCM payload encryption with Android Keystore

```

object PayloadCrypto {
    private const val KEY_ALIAS = "payload_key"
    private const val ALGORITHM = KeyProperties.KEY_ALGORITHM_AES
    private const val BLOCK_MODE = KeyProperties.BLOCK_MODE_GCM
    private const val PADDING = KeyProperties.ENCRYPTION_PADDING_NONE
    private const val TRANSFORM = "$ALGORITHM/$BLOCK_MODE/$PADDING"

    private fun getOrCreateKey(): SecretKey {
        val ks = KeyStore.getInstance("AndroidKeyStore").apply { load(null) }
        return (ks.getEntry(KEY_ALIAS, null) as? KeyStore.SecretKeyEntry)?.secretKey
            ?: KeyGenerator.getInstance(ALGORITHM, "AndroidKeyStore").apply {
                init(KeyGenParameterSpec.Builder(KEY_ALIAS,
                    KeyProperties.PURPOSE_ENCRYPT or KeyProperties.PURPOSE_DECRYPT)
                    .setBlockModes(BLOCK_MODE)
                    .setEncryptionPaddings(PADDING)
                    .setKeySize(256)
                    .build())
            }.generateKey()
    }

    fun encrypt(plaintext: ByteArray): Pair<ByteArray, ByteArray> {
        val cipher = Cipher.getInstance(TRANSFORM).apply {
            init(Cipher.ENCRYPT_MODE, getOrCreateKey())
        }
        return cipher.iv to cipher.doFinal(plaintext)
    }

    fun decrypt(iv: ByteArray, ciphertext: ByteArray): ByteArray {
        val cipher = Cipher.getInstance(TRANSFORM).apply {
            init(Cipher.DECRYPT_MODE, getOrCreateKey(), GCMParameterSpec(128, iv))
        }
        return cipher.doFinal(ciphertext)
    }
}

```

19. Exchanging Data Between Apps

Android provides multiple IPC mechanisms. Choosing the right one depends on the relationship between the apps, the data volume, and security requirements. As of Android 11+ package visibility restrictions apply — apps must explicitly declare which other packages they query.

19.1 Intents & Intent Filters

Intents are the primary loose-coupling mechanism for launching activities, delivering results, or starting services across app boundaries.

Kotlin – Sending and receiving structured data via Intent

```
// Sending app – explicit intent to a known package
val intent = Intent(Intent.ACTION_SEND).apply {
    type = "text/plain"
    putExtra(Intent.EXTRA_TEXT, "Hello from App A")
    putExtra(Intent.EXTRA_SUBJECT, "Shared content")
}
startActivity(Intent.createChooser(intent, "Share via"))

// Receiving app – declare filter in AndroidManifest.xml
// <activity android:name=".ShareReceiver" android:exported="true">
// <intent-filter>
// <action android:name="android.intent.action.SEND" />
// <category android:name="android.intent.category.DEFAULT" />
// <data android:mimeType="text/plain" />
// </intent-filter>
// </activity>

// Receiving activity
class ShareReceiver : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val sharedText = intent.getStringExtra(Intent.EXTRA_TEXT) ?: ""
        setContent { MyAppTheme { ShareScreen(sharedText) } }
    }
}

// Android 11+ – query packages before resolving implicit intents
// AndroidManifest.xml:
// <queries>
// <intent><action android:name="android.intent.action.SEND" /></intent>
// </queries>
```

19.2 Pending Intents — Secure Callbacks

PendingIntent wraps an Intent so another app or the system can execute it on your behalf. Always use FLAG_IMMUTABLE (Android 12+ requirement) and FLAG_ONE_SHOT for single-use callbacks.

Kotlin – Safe PendingIntent construction

```
// Android 12+ requires FLAG_IMMUTABLE or FLAG_MUTABLE explicitly
val pendingIntent = PendingIntent.getActivity(
```



```

context,
REQUEST_CODE,
Intent(context, DeepLinkActivity::class.java).apply {
    putExtra("notification_id", notifId)
},
PendingIntent.FLAG_IMMUTABLE or PendingIntent.FLAG_UPDATE_CURRENT
)

// For broadcast receivers
val broadcastPi = PendingIntent.getBroadcast(
context, 0,
Intent(context, AlarmReceiver::class.java),
PendingIntent.FLAG_IMMUTABLE
)

// NEVER use FLAG_MUTABLE unless the receiving component must fill in extras
// (e.g. MediaSession, Bubbles API). Document clearly why it is mutable.

```

19.3 Bound Services & AIDL

AIDL (Android Interface Definition Language) enables structured, type-safe cross-process calls. Use it only when you need synchronous two-way IPC with a different app's process. For same-process or same-app IPC, use a regular bound service.

AIDL – ICalculator.aidl

```

// src/main/aidl/com/example/ICalculator.aidl
package com.example;
interface ICalculator {
    int add(int a, int b);
    double divide(double numerator, double denominator);
}

```

Kotlin – Service-side AIDL implementation

```

class CalculatorService : Service() {
    private val binder = object : ICalculator.Stub() {
        override fun add(a: Int, b: Int): Int = a + b
        override fun divide(num: Double, den: Double): Double {
            require(den != 0.0) { "Division by zero" }
            return num / den
        }
    }
    override fun onBind(intent: Intent?): IBinder = binder
}

```

Kotlin – Client-side binding (Compose ViewModel)

```

class CalcViewModel : ViewModel() {
    private var calculator: ICalculator? = null

    private val connection = object : ServiceConnection {
        override fun onServiceConnected(name: ComponentName, binder: IBinder) {
            calculator = ICalculator.Stub.asInterface(binder)
        }
        override fun onServiceDisconnected(name: ComponentName) {
            calculator = null
        }
    }
}

```

```

}
}

fun bind(context: Context) {
    val intent = Intent().apply {
        component = ComponentName("com.example.calc", "com.example.calc.CalculatorService")
    }
    context.bindService(intent, connection, Context.BIND_AUTO_CREATE)
}

fun add(a: Int, b: Int): Int = calculator?.add(a, b) ?: error("Not bound")

override fun onCleared() {
    // Unbind happens in DisposableEffect in the composable
}
}

```

19.4 Broadcast Receivers

Broadcasts notify one or many apps about system or app events. Since Android 8+, most implicit broadcasts cannot be received by manifest-registered receivers. Always use context-registered receivers for runtime events.

Kotlin – Context-registered receiver (Compose lifecycle-aware)

```

@Composable
fun BatteryStatusBanner() {
    var isCharging by remember { mutableStateOf(false) }
    val context = LocalContext.current

    DisposableEffect(context) {
        val receiver = object : BroadcastReceiver() {
            override fun onReceive(ctx: Context, intent: Intent) {
                val status = intent.getIntExtra(BatteryManager.EXTRA_STATUS, -1)
                isCharging = status == BatteryManager.BATTERY_STATUS_CHARGING ||
                    status == BatteryManager.BATTERY_STATUS_FULL
            }
        }
        val filter = IntentFilter(Intent.ACTION_BATTERY_CHANGED)
        context.registerReceiver(receiver, filter)
        onDispose { context.unregisterReceiver(receiver) }
    }

    if (isCharging) {
        Surface(color = MaterialTheme.colorScheme.primaryContainer) {
            Text("Charging", modifier = Modifier.padding(8.dp))
        }
    }
}

// Sending a local broadcast (within same process)
LocalBroadcastManager.getInstance(context)
    .sendBroadcast(Intent("com.example.DATA_UPDATED"))

```

- Use LocalBroadcastManager for intra-app events — more efficient and secure than global broadcasts.
- For ordered broadcasts where a result is needed, use sendOrderedBroadcast().
- Protect exported broadcast receivers with custom permissions:
android.permission='com.example.MY_PERMISSION'.
- Android 13+: exported receivers MUST declare android:exported=true or false explicitly.

19.5 File Sharing via FileProvider

FileProvider grants temporary, permission-scoped URI access to files — the safe alternative to file:// URIs which are blocked by StrictMode on Android 7+.

XML – AndroidManifest.xml FileProvider declaration

```
<provider
    android:name="androidx.core.content.FileProvider"
    android:authorities="${applicationId}.fileprovider"
    android:exported="false"
    android:grantUriPermissions="true">
    <meta-data
        android:name="android.support.FILE_PROVIDER_PATHS"
        android:resource="@xml/file_paths" />
    </provider>

<!-- res/xml/file_paths.xml -->
<paths>
    <cache-path name="cached_images" path="images/" />
    <external-cache-path name="ext_cache" path="." />
    <files-path name="downloads" path="downloads/" />
</paths>
```

Kotlin – Share a file with another app

```
fun shareImageFile(file: File, context: Context) {
    val uri = FileProvider.getUriForFile(
        context,
        "${context.packageName}.fileprovider",
        file
    )
    val intent = Intent(Intent.ACTION_SEND).apply {
        type = "image/jpeg"
        putExtra(Intent.EXTRA_STREAM, uri)
        addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION) // temporary read grant
    }
    context.startActivity(Intent.createChooser(intent, "Share Image"))
}
```

20. Content Providers

Content Providers are Android's structured data-sharing mechanism. They expose relational-style data (via `ContentResolver` + `Cursor`) to other apps using `content://` URIs. The system's `Contacts`, `MediaStore`, and `CallLog` all use this API. In Jetpack Compose, you interact with them through coroutines and `Flow`, not raw callbacks.

20.1 Built-in Content Providers

Kotlin – Query MediaStore for photos

```
suspend fun queryPhotos(context: Context): List<Uri> =
    withContext(Dispatchers.IO) {
        val photos = mutableListOf<Uri>()
        val projection = arrayOf(
            MediaStore.Images.Media._ID,
            MediaStore.Images.Media.DISPLAY_NAME,
            MediaStore.Images.Media.DATE_TAKEN
        )
        val sortOrder = "${MediaStore.Images.Media.DATE_TAKEN} DESC"

        context.contentResolver.query(
            MediaStore.Images.Media.EXTERNAL_CONTENT_URI,
            projection,
            null, null, sortOrder
       )?.use { cursor ->
            val idCol = cursor.getColumnIndexOrThrow(MediaStore.Images.Media._ID)
            val nameCol = cursor.getColumnIndexOrThrow(MediaStore.Images.Media.DISPLAY_NAME)
            while (cursor.moveToNext()) {
                val uri = ContentUris.withAppendedId(
                    MediaStore.Images.Media.EXTERNAL_CONTENT_URI,
                    cursor.getLong(idCol)
                )
                photos.add(uri)
            }
        }
        photos
    }
```

Kotlin – Query Contacts (requires `READ_CONTACTS` permission)

```
suspend fun searchContacts(context: Context, query: String): List<String> =
    withContext(Dispatchers.IO) {
        buildList {
            context.contentResolver.query(
                ContactsContract.CommonDataKinds.Phone.CONTENT_URI,
                arrayOf(ContactsContract.CommonDataKinds.Phone.DISPLAY_NAME,
                    ContactsContract.CommonDataKinds.Phone.NUMBER),
                "${ContactsContract.CommonDataKinds.Phone.DISPLAY_NAME} LIKE ?",
                arrayOf("%$query%"),
                "${ContactsContract.CommonDataKinds.Phone.DISPLAY_NAME} ASC"
           )?.use { cursor ->
                while (cursor.moveToNext()) {
                    add(cursor.getString(0) + " – " + cursor.getString(1))
                }
            }
        }
    }
```

```

}
}
}
}

```

20.2 Custom Content Provider

Implement a Content Provider when you need to share structured data from your app's Room database with other apps, or with system components like widgets and search suggestions.

Kotlin – Custom ContentProvider backed by Room

```

class ItemContentProvider : ContentProvider() {

    private lateinit var db: AppDatabase

    companion object {
        const val AUTHORITY = "com.example.app.provider"
        val ITEMS_URI: Uri = Uri.parse("content://$AUTHORITY/items")
        private val uriMatcher = UriMatcher(UriMatcher.NO_MATCH).apply {
            addURI(AUTHORITY, "items", 1) // all items
            addURI(AUTHORITY, "items/#", 2) // single item by ID
        }
    }

    override fun onCreate(): Boolean {
        db = Room.databaseBuilder(context!!, AppDatabase::class.java, "app.db")
            .allowMainThreadQueries() // Required in ContentProvider – runs on Binder thread
            .build()
        return true
    }

    override fun query(
        uri: Uri, projection: Array<out String>?,
        selection: String?, selectionArgs: Array<out String>?, sortOrder: String?
    ): Cursor? = when (uriMatcher.match(uri)) {
        1 -> db.itemDao().getAllCursor()
        2 -> db.itemDao().getByIdCursor(ContentUris.parseId(uri))
        else -> null
    }

    override fun getType(uri: Uri): String = when (uriMatcher.match(uri)) {
        1 -> "vnd.android.cursor.dir/vnd.com.example.items"
        2 -> "vnd.android.cursor.item/vnd.com.example.items"
        else -> throw IllegalArgumentException("Unknown URI: $uri")
    }

    override fun insert(uri: Uri, values: ContentValues?): Uri? {
        val id = db.itemDao().insert(values!!.toItem())
        context?.contentResolver?.notifyChange(uri, null)
        return ContentUris.withAppendedId(ITEMS_URI, id)
    }

    override fun update(uri: Uri, values: ContentValues?,

```

```

selection: String?, selectionArgs: Array<out String>?): Int {
    val count = db.itemDao().update(values!!.toItem())
    context?.contentResolver?.notifyChange(uri, null)
    return count
}

override fun delete(uri: Uri, selection: String?,
selectionArgs: Array<out String>?): Int {
    val id = ContentUris.parseId(uri)
    val count = db.itemDao().deleteById(id)
    context?.contentResolver?.notifyChange(uri, null)
    return count
}
}

```

XML – Register provider in AndroidManifest.xml

```

<provider
    android:name=".data.provider.ItemContentProvider"
    android:authorities="com.example.app.provider"
    android:exported="false" <!-- true only if sharing with other apps -->
    android:readPermission="com.example.app.READ_ITEMS"
    android:writePermission="com.example.app.WRITE_ITEMS" />

```

20.3 Querying a Provider from Compose

Wrap ContentResolver queries in a Flow so the Compose UI reactively updates when the underlying data changes (via contentResolver.notifyChange).

Kotlin – Reactive ContentResolver query as Flow

```

fun ContentResolver.observeQuery(
    uri: Uri,
    projection: Array<String>? = null,
    selection: String? = null,
    selectionArgs: Array<String>? = null,
    sortOrder: String? = null
): Flow<Cursor?> = callbackFlow {
    val observer = object : ContentObserver(Handler(Looper.getMainLooper())) {
        override fun onChange(selfChange: Boolean) {
            trySend(query(uri, projection, selection, selectionArgs, sortOrder))
        }
    }
    registerContentObserver(uri, true, observer)
    trySend(query(uri, projection, selection, selectionArgs, sortOrder))
    awaitClose { unregisterContentObserver(observer) }
}

// In ViewModel
val items: StateFlow<List<Item>> = contentResolver
    .observeQuery(ItemContentProvider.ITEMS_URI)
    .map { cursor -> cursor?.toItems() ?: emptyList() }
    .flowOn(Dispatchers.IO)
    .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), emptyList())

```

20.4 CursorLoader & Paging

Kotlin – Paging 3 with ContentProvider data source

```
class ItemPagingSource(
    private val resolver: ContentResolver
) : PagingSource<Int, Item>() {

    override fun getRefreshKey(state: PagingState<Int, Item>): Int? =
        state.anchorPosition

    override suspend fun load(params: LoadParams<Int>): LoadResult<Int, Item> {
        val offset = params.key ?: 0
        return withContext(Dispatchers.IO) {
            val cursor = resolver.query(
                ItemContentProvider.ITEMS_URI.buildUpon()
                    .appendQueryParameter("limit", params.loadSize.toString())
                    .appendQueryParameter("offset", offset.toString())
                    .build(),
                null, null, null, null
            )
            val items = cursor?.use { it.toItems() } ?: emptyList()
            LoadResult.Page(
                data = items,
                prevKey = if (offset == 0) null else offset - params.loadSize,
                nextKey = if (items.isEmpty()) null else offset + params.loadSize
            )
        }
    }
}
```

20.5 Content Provider Permissions & Security

Kotlin – Grant temporary URI permissions to another app

```
// Grant read access to a specific URI to a specific package
val grantIntent = Intent().apply {
    data = specificItemUri
    flags = Intent.FLAG_GRANT_READ_URI_PERMISSION
}
context.grantUriPermission("com.other.app", specificItemUri,
    Intent.FLAG_GRANT_READ_URI_PERMISSION)

// Revoke when done
context.revokeUriPermission(specificItemUri, Intent.FLAG_GRANT_READ_URI_PERMISSION)
```

- Set `android:exported="false"` unless external apps must access your provider.
- Define separate read and write permissions using `android:readPermission` / `android:writePermission`.
- Use `android:grantUriPermissions="true"` with `FileProvider` for temporary access instead of exporting the whole provider.
- Validate and sanitize all selection arguments — treat them like SQL user input; use `?` placeholders, never string concatenation.
- Call `notifyChange()` after every insert/update/delete so observers (`ContentObserver`, `Paging`) refresh automatically.

- Test with `adb shell content query --uri content://com.example.app.provider/items` to verify the provider is accessible and returns expected columns.

Quick-Reference Summary

Area	Key Rule / Best Practice
Design	Use MaterialTheme tokens; support dark mode; define spacing tokens
Architecture	MVVM + UDF; hoist state; use StateFlow + collectAsStateWithLifecycle
Performance	Stable params; derivedStateOf; keys in LazyColumn; size images
Memory	No Context in ViewModel; cancel coroutines; unregister in onDispose
Navigation	Type-safe routes; single NavHost; pass lambdas not NavController
Responsive UI	WindowSizeClass; NavigationRail on Expanded; edge-to-edge insets
ProGuard / R8	isMinifyEnabled=true; keep data models; upload mapping to Play
Security	EncryptedSharedPreferences; certificate pinning; FLAG_SECURE; Keystore
Accessibility	Semantics roles; 48dp touch targets; null for decorative images; 4.5:1 contrast
Testing	Unit-test VMs; ComposeTestRule for UI; Paparazzi for screenshots
Hilt DI	@HiltAndroidApp; @HiltViewModel; hiltViewModel() in composables
Animations	animate*AsState for single values; AnimatedVisibility; updateTransition
Networking	Repository pattern; Flow from suspend; Dispatchers.IO
Data Storage	DataStore over SharedPreferences; Room + Flow DAO; never plain files for secrets
Naming Conventions	PascalCase classes; camelCase functions; SCREAMING_SNAKE constants
Composable Naming	PascalCase for UI nodes; camelCase for composables returning values
Modifier Pattern	Always accept modifier param; pass through unmodified; size then padding
Parameter Order	Required data > modifier > optional config > onX callbacks > content slot
Kotlin — Null Safety	No !! outside tests; use ?.let, ?: elvis, requireNotNull
Kotlin — Flows	combine, debounce, retry, flatMapLatest; stateIn with WhileSubscribed(5000)
Error Handling	Sealed Result types; catch at data layer; typed errors to UI via ViewModel
Logging	Timber only; no PII; strip debug logs in release via ProGuard
Gradle	Version catalogs; parallel=true; caching=true; config-cache=true
Modularisation	Feature modules; thin :app; features never depend on each other
CI/CD	GitHub Actions: ktlint + detekt + unit tests + AAB deploy to Play
Battery Life	WorkManager + Constraints (Wi-Fi, charging); avoid exact alarms; batch sensor reads
Doze / Standby	Design for deferred work; FCM for time-critical notifications; test with adb
Foreground Service	Declare foregroundServiceType; show persistent notification; use START_STICKY

TLS / HTTPS	cleartextTrafficPermitted=false; TLS 1.2+ only; strong cipher suites
Cert Pinning	Pin primary + backup; set expiry; handle SSLPeerUnverifiedException gracefully
OAuth / Tokens	AppAuth PKCE flow; refresh interceptor; store tokens in EncryptedSharedPreferences
Payload Encryption	AES-256-GCM via Android Keystore; keys never leave secure enclave
Intents	Use FLAG_IMMUTABLE PendingIntents; declare <queries> for Android 11+
Broadcasts	Context-register in DisposableEffect; LocalBroadcastManager for intra-app
FileProvider	Share files via content:// URI + FLAG_GRANT_READ_URI_PERMISSION; never file://
Content Providers	android:exported=false by default; separate read/write permissions; notifyChange()
Provider Querying	Wrap in callbackFlow + ContentObserver for reactive Compose UI
Provider Security	Sanitize selections with ? placeholders; grant temporary URI permissions